Architecture Patterns for AI Applications

Module 14 — Business AI in Practice · Notebook 44

Prof. Dr. Christoph Weisser

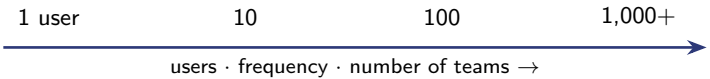
HSBI

Summer Semester 2026

Contents

Outline

Architecture is the answer to a *traffic* question



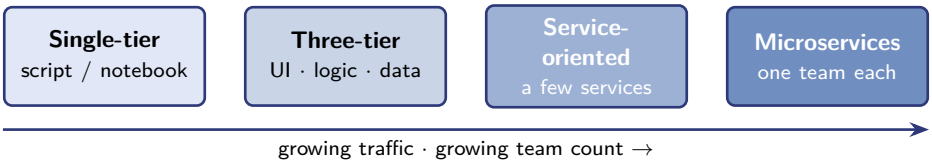
Ask three things before you choose

- **How many users**, and growing how fast?
- **Who invokes it** — a human, a schedule, or another service?
- **How many teams** will own and ship pieces of it?

Architecture is not sophistication — it is matching *structure* to *load*.

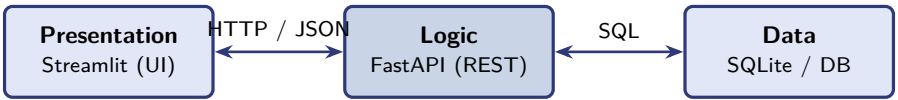
Outline

Four patterns of growing complexity



Read left to right as *growing load*, not "primitive to advanced". The simplest workable choice wins.

Three-tier — the workhorse for interactive AI features



- Each tier can change independently; the **REST contract** is the boundary.
- The API is reusable — a second UI, a CLI, or another team can call it.
- *You build exactly this* in **Notebook 28 (POC 2)**.

The over-engineering trap

Microservices for a 5-user pilot

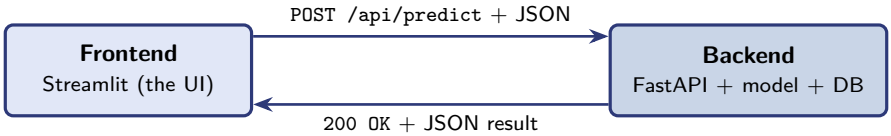
Microservices became fashionable around 2015; many teams adopted them *before* they had the team size or traffic to justify the operational cost. The right unit of a service is a *capability* — not every Python module.

The rule

Pick the architecture you'll have *outgrown in 12 months*, not the one you'll *need in 36*. Premature complexity kills more projects than migrations do.

Outline

Frontend ↔ backend: the request/response cycle



- The frontend *never* touches the database — it only calls the backend over **HTTP**.
- A request names an **endpoint** (URL path) + a **verb** (`GET` read, `POST` send data) + a JSON **body**.
- The backend validates, runs the logic / model, and returns **JSON** + a **status code** (200 ok · 4xx client error · 5xx server error).

The backend — one FastAPI endpoint

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Customer(BaseModel): # validates the incoming JSON
    tenure: int
    monthly_charges: float

@app.post("/api/predict") # endpoint path + HTTP verb
def predict(c: Customer):
    p = model.predict_proba([[c.tenure, c.monthly_charges]])[0][1]
    return {"churn_probability": round(float(p), 3)}
```

Pydantic **validates** the body automatically; returning a dict is serialised to JSON; interactive docs appear free at `/docs`.

The frontend — calling the backend

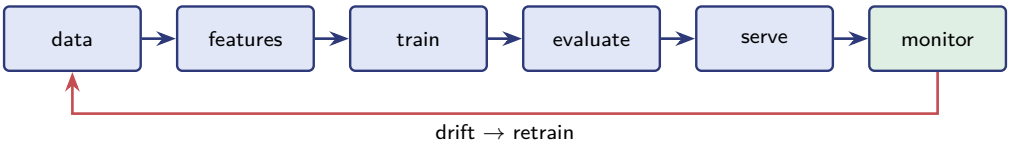
```python
import requests, streamlit as st

resp = requests.post(
    "http://localhost:8000/api/predict",
    json={"tenure": 12, "monthly_charges": 79.9},
    timeout=10,
)
resp.raise_for_status() # turn 4xx / 5xx into an error
st.metric("Churn risk", resp.json()["churn_probability"])
```

- The frontend is just an **HTTP client** — the same `requests` pattern as NB 12.
- **CORS:** a browser blocks cross-origin calls — add FastAPI's `CORSMiddleware` so the UI (port 8501) may call the API (port 8000).

You build this end-to-end in **Notebook 28 (POC 2)** — Streamlit + FastAPI + SQLite.

Outline

The cross-cutting ML / AI pipeline



What pure-software patterns don't capture

- **Training data is part of the artefact** — different snapshot, different product.
- **Drift exists** — 92 % in March is not 92 % in October.
- **The feedback loop has economic cost** — each prediction nudges the next training batch.

Outline

Choosing the right size

Smells that you've outgrown your current shape

- "Can we run this nightly?" → add scheduling (single-tier job).
- A non-developer needs to use it → add a UI tier (three-tier).
- A second team wants to build on it → expose it as a service.

Two opposite mistakes

Over-engineering (microservices for a pilot) and under-engineering (a 200-user system held together by one notebook).

Outline

How the course maps to these patterns

Pattern	**Where it appears in the course**
Single-tier (script / notebook)	Almost every notebook in the course, by default
Scheduled single-tier	NB 40 — scheduling & orchestration
Three-tier with AI in the middle	NB 28 (POC 2) and the NB 42 capstone
Service-oriented	`llm_providers.py` — pluggable providers
Microservices	*patterns only* — too big for a teaching course
ML pipeline	NB 14–16 + NB 39–40 — train / evaluate / serve

You *build* the three-tier and ML-pipeline shapes in **Notebook 28**; the rest you meet as patterns here.